```
.AddCookie(options =>
{
    options.LoginPath = "/login";
});
```

Finally, you amend the `OnGetAsync` handler in the home page `IndexModel` to return a `ChallengeResult` if the user is not authenticated. You also need to add a `using` directive for `Microsoft.AspNetCore.Mvc`.

Listing 9.8 Adding a `ChallengeResult` to the `OnGetAsync` handler

```
public async Task<IActionResult> OnGetAsync()      ❶
{
    if (User.Identity.IsAuthenticated)             ❷
    {
        Cities = await _cityService.GetAllAsync();
        return Page();
    }
    return Challenge();                            ❸
}
```

❶ Change the method to return Task<IActionResult>.

❷ Check to see if the user is authenticated before showing the content.

❸ If not, issue a challenge.

In addition to the Razor page, the `PageModel` class exposes the current user via a `User` property. You will access that to establish whether the current user is authenticated, and if not, you will use the `Challenge` helper method to return a new `ChallengeResult`.

Now that everything is in place, you can run the application. You should automatically be redirected to the configured login path when the application launches, but notice that the URL in the browser address bar also includes a query string: `/login?ReturnUrl=%2F`. The `ReturnUrl` query string value represents the page that issued the challenge—in this case, the home page. `%2F` is a URL-encoded forward slash. Once you have authenticated by entering a username and submitting the login form, the `SignInAsync` method will